

Build a RESTful API With Flask, MongoDB, and Python

- Data transformation between the database and a JSON representation.
- Data validation.
- Pagination.
- Error-handling.

Prerequisites

- Python 3.8 or above
- A MongoDB Atlas cluster. Follow the "[Get Started with Atlas](#)" guide to create your account and MongoDB cluster. Keep a note of your database username, password, and [connection string](#) as you will need those later.

This is an *advanced* guide, so it'll cover a whole bunch of different libraries which can be brought together to build a declarative Restful API server on top of MongoDB. I won't cover repeating patterns in the codebase, so if you want to build the whole thing, I recommend checking out the source code, which is all [on GitHub](#).

It won't cover the basics of Python, Flask, or MongoDB, so if that's what you're looking for, I recommend checking out the following resources before tackling this post:

- [Think Python](#)
- [The Python & MongoDB Quickstart Series](#)
- [Flask Tutorial](#)
- [Pydantic Documentation](#)

Getting Started

Begin by cloning the sample code source [from GitHub](#). There are four top-level directories:

- [actix-cocktail-api](#): You can ignore this for now.
- [data](#): This contains an export of my cocktail data. You'll import this into your cluster in a moment.
- [flask-cocktail-api](#): The code for this blog post.
- [test_scripts](#): A few shell scripts that use curl to test the HTTP interface of the API server.

There are more details in the [GitHub repo](#), but the basics are: Install the project with your virtualenv active:

```
pip install -e .
```

Next, you should import the data into your cluster. Set the environment variable `$MONGO_URI` to your cluster URI. This environment variable will be used in a moment to import your data, and also by the Flask app. I use `direnv` to configure this, and put the following line in my `.envrc` file in my project's directory:

```
export MONGO_URI="mongodb+srv://USERNAME:PASSWORD@cluster0-  
abcde.azure.mongodb.net/cocktails?retryWrites=true&w=majority"
```

Note that your database must be called "cocktails," and the import will create a collection called "recipes." After checking that `$MONGO_URI` is set correctly, run the following command:

```
mongoimport --uri "$MONGO_URI" --file ./recipes.json
```

Now you should be able to run the Flask app from the `flask-cocktail-api` directory:

```
FLASK_DEBUG=true FLASK_APP=cocktailapi flask run
```

(You can run `make run` if you prefer.)

Check the output to ensure it is happy with the configuration, and then in a different terminal window, run

the `list_cocktails.sh` script in the `test_scripts` directory.

It should print something like this:

```
{  
  "_links": {  
    "last": {
```

```
"href": "http://localhost:5000/cocktails/?page=5"
},
"next": {
  "href": "http://localhost:5000/cocktails/?page=5"
},
"prev": {
  "href": "http://localhost:5000/cocktails/?page=3"
},
"self": {
  "href": "http://localhost:5000/cocktails/?page=4"
},
"recipes": [
  {
    "_id": "5f7daa198ec9dfb536781b0d",
    "date_added": null,
    "date_updated": null,
    "ingredients": [
      {
        "name": "Light rum",
        "quantity": {
          "unit": "oz",
        }
      },
      {
        "name": "Grapefruit juice",
        "quantity": {
          "unit": "oz",
        }
      },
      {
        "name": "Bitters",
        "quantity": {
          "unit": "dash",
        }
      }
    ],
    "instructions": [
      "Pour all of the ingredients into an old-fashioned glass almost filled with ice cubes",
      "Stir well."
    ],
    "name": "Monkey Wrench",
    "slug": "monkey-wrench"
  },

```

```
]
...
```

Breaking it All Down

The code is divided into three submodules.

- `__init__.py` contains all the Flask setup code, and defines all the HTTP routes.
- `model.py` contains all the Pydantic model definitions.
- `objectid.py` contains a Pydantic field definition that I stole from the [Beanie](#) object-data mapper for MongoDB.

I mentioned earlier that this code makes use of several libraries:

- [PyMongo](#) and [Flask-PyMongo](#) handle the connection to the database. Flask-PyMongo specifically wraps the database collection object to provide a convenient `find_one_or_404` method.
- [Pydantic](#) manages data validation, and some aspects of data transformation between the database and a JSON representations.
- along with a single function from [FastAPI](#).

Data Validation and Transformation

When building a robust API, it's important to validate all the data passing into the system. It would be possible to do this using a stack of `if/else` statements, but it's much more effective to define a schema declaratively, and to allow that to programmatically validate the data being input.

I used a technique that I learned from [Beanie](#), a new and neat ODM that I unfortunately couldn't practically use on this project, because Beanie is async, and Flask is a blocking framework.

Beanie uses [Pydantic](#) to define a schema, and adds a custom Field type for ObjectId.

```
# model.py
class Cocktail(BaseModel):
    id: Optional[PydanticObjectId] = Field(None, alias="_id")
    slug: str
```

```

name: str
ingredients: List[Ingredient]
instructions: List[str]
date_added: Optional[datetime]
date_updated: Optional[datetime]
def to_json(self):
    return jsonable_encoder(self, exclude_none=True)
def to_bson(self):
    data = self.dict(by_alias=True, exclude_none=True)
    if data["_id"] is None:
        data.pop("_id")
    return data

```

This `Cocktail` schema defines the structure of a `Cocktail` instance, which will be validated by Pydantic when instances are created. It includes another embedded schema for `Ingredient`, which is defined in a similar way. I added convenience functions to export the data in the `Cocktail` instance to either a JSON-compatible `dict` or a BSON-compatible `dict`. The differences are subtle, but BSON supports native `ObjectId` and `datetime` types, for example, whereas when encoding as JSON, it's necessary to encode `ObjectId` instances in some other way (I prefer a string containing the hex value of the id), and `datetime` objects are encoded as ISO8601 strings. The `to_json` method makes use of a function imported from FastAPI, which recurses through the instance data, encoding all values in a JSON-compatible form. It already handles `datetime` instances correctly, but to get it to handle `ObjectId` values, I extracted some [custom field](#) code from Beanie, which can be found in `objectid.py`.

The `to_bson` method doesn't need to pass the `dict` data through `jsonable_encoder`. All the types used in the schema can be directly saved with PyMongo. It's important to set `by_alias` to `True`, so that the key for `_id` is just that, `_id`, and not the schema's `id` without an underscore.

```

# objectid.py
class PydanticObjectId (ObjectId):

```

```

"""
ObjectId field. Compatible with Pydantic.
"""

@classmethod
def __get_validators__(cls):
    yield cls.validate

@classmethod
def validate(cls, v):
    return PydanticObjectId(v)

@classmethod
def __modify_schema__(cls, field_schema: dict):
    field_schema.update(
        type="string",
        examples=["5eb7cf5a86d9755df3a6c593", "5eb7cfb05e32e07750a1756a"],
    )
ENCODERS_BY_TYPE[PydanticObjectId] = str

```

This approach is neat for this particular use-case, but I can't help feeling that it would be limiting in a more complex system. There are many [patterns for storing data in MongoDB](#). These often result in storing data in a form that is optimal for writes or reads, but not necessarily the representation you would wish to export in an API.

What is a Slug?

Looking at the schema above, you may have wondered what a "slug" is ... well, apart from a slimy garden pest.

A slug is a unique, URL-safe, mnemonic used for identifying a document. I picked up the terminology as a Django developer, where this term is part of the framework. A slug is usually derived from another field. In this case, the slug is derived from the name of the cocktail, so if a cocktail was called "Rye Whiskey Old-Fashioned," the slug would be "rye-whiskey-old-fashioned."

In this API, that cocktail could be accessed by sending a GET request to the `/cocktails/rye-whiskey-old-fashioned` endpoint.

I've kept the unique `slug` field separate from the auto-assigned `_id` field, but I've provided both because the slug could change if the name of the cocktail was tweaked, in which case the `_id` value would provide a constant identifier to look up an exact document.

In the Rust version of this code, I was nudged to use a different approach. It's a bit more verbose, but in the end I was convinced that it would be more powerful and flexible as the system grew.

Creating a New Document

Now I'll show you what a single endpoint looks like, first focusing on the "Create" endpoint, that handles a POST request to `/cocktails` and creates a new document in the "recipes" collection. It then returns the document that was stored, including the newly unique ID that MongoDB assigned as `_id`, because this is a RESTful API, and that's what RESTful APIs do.

```
@app.route("/cocktails/", methods=["POST"])
def new_cocktail():
    raw_cocktail = request.get_json()
    raw_cocktail["date_added"] = datetime.utcnow()
    cocktail = Cocktail(**raw_cocktail)
    insert_result = recipes.insert_one(cocktail.to_bson())
    cocktail.id = PydanticObjectId(str(insert_result.inserted_id))
    print(cocktail)
    return cocktail.to_json()
```

This endpoint modifies the incoming JSON directly, to add a `date_added` item with the current time. It then passes it to the constructor for our Pydantic schema. At this point, if the schema failed to validate the data, an exception would be raised and displayed to the user.

After validating the data, `to_bson()` is called on the `Cocktail` to convert it to a BSON-compatible dict, and this is directly passed to PyMongo's `insert_one` method. There's no way to get PyMongo to return the document that was just inserted in a single operation (although an upsert using `find_one_and_update` is similar to just that).

After inserting the data, the code then updates the local object with the newly-assigned `id` and returns it to the client.

Reading a Single Cocktail

Thanks to `Flask-PyMongo`, the endpoint for looking up a single cocktail is even more straightforward:

```
@app.route("/cocktails/<string:slug>", methods=["GET"])
def get_cocktail(slug):
    recipe = recipes.find_one_or_404({"slug": slug})
    return Cocktail(**recipe).to_json()
```

This endpoint will abort with a 404 if the slug can't be found in the collection. Otherwise, it simply instantiates a Cocktail with the document from the database, and calls `to_json` to convert it to a dict that Flask will automatically encode correctly as JSON.

Listing All the Cocktails

This endpoint is a monster, and it's because of pagination, and the links for pagination. In the sample data above, you probably noticed the `_links` section:

```
"_links": {
  "last": {
    "href": "http://localhost:5000/cocktails/?page=5"
  },
  "next": {
    "href": "http://localhost:5000/cocktails/?page=5"
  },
  "prev": {
    "href": "http://localhost:5000/cocktails/?page=3"
  },
  "self": {
    "href": "http://localhost:5000/cocktails/?page=4"
  }
},
```

This `_links` section is specified as part of the [HAL \(Hypertext Application Language\)](#) specification. It's a good idea to follow a standard for pagination data, and I didn't feel like inventing something myself!

And here's the code to generate all this. Don't freak out.

```
@app.route("/cocktails/")
def list_cocktails():
    """
    GET a list of cocktail recipes.
    The results are paginated using the `page` parameter.
    """
    page = int(request.args.get("page", 1))
    per_page = 10 # A const value.
    # For pagination, it's necessary to sort by name,
```

```

# then skip the number of docs that earlier pages would have displayed,
# and then to limit to the fixed page size, ``per_page``.
cursor = recipes.find().sort("name").skip(per_page * (page - 1)).limit(per_page)
cocktail_count = recipes.count_documents({})
links = {
    "self": {"href": url_for(".list_cocktails", page=page, _external=True)},
    "last": {
        "href": url_for(
            ".list_cocktails", page=(cocktail_count // per_page) + 1, _external=True
        )
    },
}
# Add a 'prev' link if it's not on the first page:
if page > 1:
    links["prev"] = {
        "href": url_for(".list_cocktails", page=page - 1, _external=True)
    }
# Add a 'next' link if it's not on the last page:
if page - 1 < cocktail_count // per_page:
    links["next"] = {
        "href": url_for(".list_cocktails", page=page + 1, _external=True)
    }
return {
    "recipes": [Cocktail(**doc).to_json() for doc in cursor],
    "_links": links,
}

```

Although there's a lot of code there, it's not as complex as it may first appear. Two requests are made to MongoDB: one for a page-worth of cocktail recipes, and the other for the total number of cocktails in the collection. Various calculations are done to work out how many documents to skip, and how many pages of cocktails there are. Finally, some links are added for "prev" and "next" pages, if appropriate (i.e.: the current page isn't the first or last.) Serialization of the cocktail documents is done in the same way as the previous endpoint, but in a loop this time.

The update and delete endpoints are mainly repetitions of the code I've already included, so I'm not going to include them here. Check them out in the [GitHub repo](#) if you want to see how they work.

Error Handling

Nothing irritates me more than using a JSON API which returns HTML when an error occurs, so I was keen to put in some reasonable error handling to avoid this happening.

After Flask set-up code, and before the endpoint definitions, the code registers two error-handlers:

```
@app.errorhandler(404)
def resource_not_found(e):
    """
    An error-handler to ensure that 404 errors are returned as JSON.
    """
    return jsonify(error=str(e)), 404

@app.errorhandler(DuplicateKeyError)
def resource_not_found(e):
    """
    An error-handler to ensure that MongoDB duplicate key errors are returned as JSON.
    """
    return jsonify(error=f"Duplicate key error."), 400
```

The first error-handler intercepts any endpoint that fails with a 404 status code and ensures that the error is returned as a JSON dict.

The second error-handler intercepts a `DuplicateKeyError` raised by any endpoint, and does the same thing as the first error-handler, but sets the HTTP status code to "400 Bad Request."

As I was writing this post, I realised that I've missed an error-handler to deal with invalid Cocktail data. I'll leave implementing that as an exercise for the reader! Indeed, this is one of the difficulties with writing robust Python applications: Because exceptions can be raised from deep in your stack of dependencies, it's very difficult to comprehensively predict what exceptions your application may raise in different circumstances.

This is something that's very different in Rust, and even though, as you'll see, error-handling in Rust can be verbose and tricky, I've started to love the language for its insistence on correctness.

Best Practices for Using Flask and MongoDB

First, use the right driver

One of the most crucial steps is to ensure you're using the correct driver. You have two choices of PyMongo or Motor. Both of these are recommended and developed by MongoDB. Pick [Motor](#) if you're building a Flask application with asyncio routes, and choose [PyMongo](#) if you are building a more traditional non-async Flask application.

To download PyMongo, just use this simple command:

```
1 pip install pymongo
```

Or you can install Motor using:

```
1 pip install motor
```

Flask!

Flask is one of the most popular frameworks out there for Python developers. It integrates fantastically with MongoDB. Here are some tips and tricks to help you out when building anything from simple projects to highly complex web applications with the two platforms.

Keep your secrets a secret

We want to keep our sensitive information properly protected when we're using Flask. While there are various ways of doing this, let's go over two of the most secure methods for saving your environment variables and API keys.

Dotenv

The first way is through the Python library `dotenv`. This will include downloading the `python-dotenv` package via pip, ensuring that you're in an activated virtual environment:

```
1 pip install python-dotenv
```

From here, you'll create a `.env` file in the root of your project directory to contain any and all environment variables, such as your MongoDB connection string:

```
1 # MongoDB connection_string
  [help](https://www.mongodb.com/docs/manual/reference/connection-string/)
2 MYAPP_CONNECTION_STRING=mongodb_connection_string_here
```

It's crucial to not commit this file! To ensure the file isn't committed, you can add your `.env` file to your project's `.gitignore` file or your global `.gitignore` file. Learn more about [keeping your .env file private](#).

You can import `load_dotenv` from the `dotenv` package, using `from dotenv import load_dotenv`. Calling the `load_dotenv` function will then copy all of the configuration from your `.env` file into your Flask application's process. You can then use `app.config.from_prefixed_env()` to import any environments that start with a certain prefix into your Flask application. Some sample skeleton code is shown below:

```
1 from dotenv import load_dotenv
2
3
4 load_dotenv()
5 app = Flask(__name__)
6 app.config.from_prefixed_env("MYAPP")
7 client = MongoClient(app.config['CONNECTION_STRING'])
```

You can access your secrets using the name of the variable without the "MYAPP_" prefix. This is where the app loads configuration from any environment variables that begin with "MYAPP_". In this case, a single configuration option is provided, `CONNECTION_STRING`.

The `dotenv` library doesn't override any existing environment variables, so you now have the choice of storing config values in

the `.env` file or overriding them by setting environment variables directly.

Flask's configuration is very flexible, and you can find out more about it from the [official documentation](#).

How do I connect to MongoDB?!

Once you've properly loaded your connection string, you can add in some code in the heart of your application, which is sometimes called the `app.py` file, to properly connect to the database and collection of your choosing, or to create a database and collection in the cluster of your choice. Let's run through how to do this step with some skeleton code:

```
1 mongo_client = MongoClient(app.config["CONNECTION_STRING"])
2
3
4 # access your database and collection from Atlas
5 database = mongo_client.get_database("anaiyamovies")
6 collection = database.get_collection("movies")
7
8
9 # instantiating new object with `__name__`
10 app = Flask(__name__)
```

What you need to know about the code above is that it doesn't contact MongoDB at all. It won't try to connect to your database cluster until you modify data in your cluster or obtain some data from your cluster. The database is named "anaiyamovies" while the collection is named "movies." Now, using either CRUD operations inside of our Flask application or simple MongoDB operators, we can write to our collection to add in documents, delete documents, change documents, or even update our docs!

How can I know if I'm actually connected to my MongoDB cluster?

Because PyMongo is designed to be lazy, it doesn't attempt to connect to your cluster when MongoClient is initialized. This can lead to surprising behavior when you eventually try to query your database, if it turns out that you can't connect. For this reason, it's good practice to validate your database connection when you create a MongoClient.

There are various ways to check if you are properly connected to your MongoDB cluster when in the process of creating your Flask application, but one of the simplest ways is to send over a `ping` command and just check the response! Let's go through how to do this.

```
1  # our imports go here
2  from pymongo import MongoClient
3  app = Flask(__name__)
4  app.config.from_prefixed_env("MYAPP")
5
6
7  def init_mongo_client(app: Flask):
8  try:
9  connection_string = "connection string here with user + pass"
10 mongo_client = MongoClient(connection_string)
11
12
13 database = mongo_client['anaiyamovies']
14 result = mongo_client.admin.command('ping')
15
16 if int(result.get('ok')) == 1:
17 print("Connected")
18 else:
19 raise Exception("Cluster ping returned OK != 1")
20 app.mongo_client = mongo_client
21
22
23 init_mongo_client(app)
```

If this function can't connect to your MongoDB cluster, this will raise an exception and fail, which is good because we want to make sure we are starting up Flask correctly. If everything's okay, it will add a `mongo_client` attribute to your Flask app that your routes can use to access your MongoDB cluster.

It's always good practice to validate your database connection when your application starts up, so that you are aware of any problems with your deployment configuration at the first opportunity.

By running a `ping` test, you can easily see if you're properly connected to the cluster you want to work with!

Encode an ObjectId correctly in JSON

By default, every document contains an ObjectId which is a type unique to BSON. This means they cannot be directly converted to JSON, so we need a workaround, such as encoding them as a string.

```
1  from bson import json_util
2  json_util.dumps(object_with_objectid)
1  from flask import Flask, Response
2  from bson import json_util
3  from typing import Mapping, Any
4
5  from dotenv import load_dotenv
6  from pymongo import MongoClient
7
8  load_dotenv()
9
10 app = Flask(__name__)
11 app.config.from_prefixed_env("MYAPP")
12
13 def init_mongodb_client(app: Flask):
14     client = MongoClient(app.config [connection_string])
15     app.db = client.get_database()
16
17     init_mongodb_client(app)
18
19
20 def bson_to_json(bson: Mapping[str, Any]) ->
    Response:
21     return Response(
22         json_util.dumps(bson),
23         content_type="application/json",
24     )
25
26
27 @app.get("/")
28 def index():
29     return bson_to_json(app.db.find_one())
```

This code will handle your ObjectIds and convert them into a string format that is easier to use and deal with in your project. Bear in mind that **BSON contains other types** that cannot be directly converted to JSON. You may also want to consider those when writing your Flask application.

Remember to model your data correctly

When you're using MongoDB, generally, it's crucial to model your data correctly so it's as efficient as possible when using various operations. To learn more about this, please check out our [Advanced Schema Design Patterns](#) course available in MongoDB University or our [Building With Patterns](#) series.

Conclusion

Hopefully, these tips and tricks are helpful for using Flask with MongoDB. Through this article, you've learned a variety of best practices that will set you up for success while integrating Flask with your MongoDB Atlas cluster and creating some fantastic applications.